

Auditoría Técnica y Master Action Plan — Cerebro Económico

NLA

Fecha: 2026-06-24 · **Auditor:** Claude (Opus 4.8) · **Alcance de la auditoría:** solo lectura

Repo: github.com/nicolasr-uner/dashboard-economico · **Clasificación:** Tipo 1 (dashboard público liviano) con auditoría de seguridad reforzada (matices Tipo 2)

> Nota metodológica: los Pasos 0-3 (identificación, reconocimiento, código, calidad/seguridad) se ejecutaron en modo solo lectura. La cobertura de datos se **midió en vivo** sobre `db.sqlite3` el 2026-06-24 (consulta `SELECT` de solo lectura), no se estimó del informe obsoleto de abril.

1. Resumen Ejecutivo

Qué es (según lo encontrado): plataforma de inteligencia macroeconómica multi-país (Colombia, México, Brasil, Ecuador + Global) construida sobre **Streamlit** (UI) + **SQLAlchemy** (ORM) + conectores `httpx` / `BeautifulSoup` a bancos centrales y organismos (BCB, FRED, Banxico, BanRep, XM, World Bank, BCE, IMF) + modelos `statsmodels` (Holt-Winters / ARIMA / Ensemble). Convive un andamiaje **Django legacy** (apps `core` / `scraper` / `ai_engine`) que **está muerto y aislado** (ningún archivo del stack vivo lo importa). **Tipo 1 + matices Tipo 2.**

Semáforo global: 🟡 (avanzando pero estancado operativamente).

El producto central (datos + dashboard) está **mucho más maduro de lo que decía la documentación**, pero hay tres frenos para producción: datos sin refrescar 63 días, cero pruebas, y deuda legacy + despliegue sin confirmar.

Complejidad estimada: ~70 % (rango 65-75 %).

Metodología (ponderada por componente; el % global pondera madurez de cada bloque, densidad de TODOs, cobertura de tests, estado de deploy y si corre):

Componente	Estado	Madurez
Ingesta de datos / cobertura	92,4 % cobertura (208/225 vars con datos, 79.624 filas) PERO fresca nula desde 2026-04-22	~70 % efectivo
Conectores	9/9 implementados (verificado: <code>bce.py</code> e <code>imf_weo.py</code> NO son stubs)	~90 %
UI dashboard	Monolito <code>streamlit_app.py</code> 100 % funcional; migración multipágina al 40 % (2/5 páginas reales, 3 stubs)	~70 %
Proyecciones estadísticas	Implementadas; se calculan en vivo (0 filas <code>PROJECTION</code> persistentes)	~80 %
Capa IA (chatbot/analyzer)	Implementada pero infrutilizada (<code>ai_analysis_log</code> = 1 fila)	~50 %
Pruebas automatizadas	0 reales	0 %
Linters / formatters	Ninguno configurado	0 %
CI/CD + despliegue	Backfill diario existe pero caído ; deploy a Streamlit Cloud no confirmado ni automatizado	~40 %
Seguridad / repo hygiene	Secrets bien aislados, pero <code>db.sqlite3</code> + <code>memoria_chat.txt</code> commiteados y credenciales hardcodeadas en <code>docker-compose.yml</code>	~60 %
Limpieza arquitectónica	Stack Django legacy aún presente; doble-ORM	~30 %

Qué falta esencialmente para producción (una frase): reparar el pipeline de ingesta (los datos llevan 63 días sin actualizarse), confirmar y automatizar el despliegue (Streamlit Cloud), y eliminar la deuda legacy (Django + higiene de repo + un mínimo de pruebas y linting).

2. Auditoría Técnica

2.1 Stack y Arquitectura (real)

- Vivo:** `streamlit_app.py` (~1.389 líneas, 87 KB) + estructura multipágina `_Home.py` + `pages/` (duplicación de endpoint). Datos vía `data/database.py` → `models/schema.py` (SQLAlchemy) → `db.sqlite3` (local) o PostgreSQL/Supabase (`DATABASE_URL`). Ingesta

orquestrada por `data/agent.py` (`VariableAgent`) usando `connectors/registry.py` → `connectors/*.py` .

- **Legacy/muerto (aislado):** `manage.py` , `config/` (Django settings/urls/wsgi/asgi), apps `core / scraper / ai_engine` , `templates/` , `static/` , y orquestación Celery/Redis en `docker-compose.yml` + `scheduler/` . **Verificado:** ningún archivo del stack Streamlit importa Django.
- **Entrypoints:** ambigüedad real entre `streamlit_app.py` (monolito) y `🏠_Home.py` (multipágina). Hay que fijar el canónico para Streamlit Cloud.

2.2 Estado de Calidad (verificado)

- **Pruebas: 0 reales.** `core/scraper/ai_engine/tests.py` son stubs de `startapp` ; `test_apis_temp.py` es ad-hoc (gitignored). Sin `pytest.ini` , `.coveragerc` , ni `[tool.pytest]` .
- **Linters/formatters: ninguno** (sin `ruff` / `black` / `flake8` / `isort` / `pre-commit` / `.editorconfig`).
- **Build/run:** se ejecuta con `streamlit run streamlit_app.py` . `Dockerfile` sin `CMD` / `EXPOSE` . `Procfile` (Heroku) y `docker-compose.yml` apuntan a Django/Celery → **rotos/abandonados** (ver 2.4).

2.3 Estado de Datos (medido en vivo, 2026-06-24)

Indicador	Valor
Variables definidas / activas	225 / 225
Variables con datos	208 (92,4 %)
Filas <code>fact_timeseries</code>	79.624 (<code>REAL_OFFICIAL</code> 78.258 · <code>ESTIMATION</code> 1.366 · <code>PROJECTION</code> 0)
Frescura (máx. fecha <code>REAL_OFFICIAL</code>)	2026-04-22 → 63 días de retraso ●
Cobertura por país	CO 96/99 · BR 36/37 · MX 28/33 · EC 24/31 · Global 24/25
<code>dim_consensus_forecast</code>	158
<code>project_financials</code>	0 (ingestor Excel no pobló la DB)
<code>ai_analysis_log</code>	1 (IA casi sin uso)

> El `docs/audit_report.md` (abril) reportaba ~13 % de cobertura: está **obsoleto** y subestima masivamente el proyecto.

2.4 Deuda Técnica

- **Doble-ORM sobre las mismas entidades** (Django `core/models.py` vs SQLAlchemy `models/schema.py`); `db.sqlite3` contiene ambas familias de tablas (`core_*` / `auth_*` / `django_*` y `dim_*` / `fact_timeseries`).
- **Stack Django legacy completo** sin servir (apps, urls, views, templates, static, manage.py).
- **Migración multipágina a medias:** 3/5 páginas son stubs "en construcción".
- `requirements.txt` **incompleto para los paths legacy:** sin Django/DRF/Celery/Redis → confirma que Heroku/Docker no son el target real. Versiones laxas (`>=`), sin lockfile.
- `imf_weo.py` **con interfaz divergente:** funciones a nivel de módulo (no hereda `BaseConnector` , sin `fetch_series`) → revisar integración con `registry.py` .
- **103 variables con `api_provider = NULL`** (100 con datos): etiquetado de procedencia incompleto.

2.5 Seguridad (críticos marcados)

#	Hallazgo	Sev.	Evidencia
S1	<code>db.sqlite3</code> (9,9 MB) commitado y NO cubierto por <code>.gitignore</code> (solo ignora <code>-wal</code> / <code>-shm</code>)	●	<code>git ls-files</code>
S2	<code>memoria_chat.txt</code> (54 KB, log de chat de desarrollo) commitado	●	<code>git ls-files</code>
S3	<code>docker-compose.yml</code> commitado con credenciales hardcodeadas (<code>dashboard_user</code> / <code>dashboard_password</code>)	●	<code>docker-compose.yml</code>
S4	<code>DATABASE_URL</code> de Supabase con contraseña embebida en <code>.env</code> / <code>secrets.toml</code> (gitignored, no en git) → rotar por precaución	⚠	<code>.env</code> , <code>.streamlit/secrets.toml</code>
S5	<code>.streamlit/config.toml</code> : <code>enableXsrfProtection=false</code> , <code>enableCORS=false</code>	●	<code>.streamlit/config.toml</code>
S6	<code>.env</code> : <code>SECRET_KEY="django-insecure-..."</code> , <code>DEBUG=True</code> (solo afecta Django legacy)	●	<code>.env</code>

#	Hallazgo	Sev.	Evidencia
—	 <code>.env</code> , <code>secrets.toml</code> , <code>*.xlsx</code> (Exagon/Ruitoque) NO trackeados; sin API keys hardcodeadas en <code>.py</code>		verificado

2.6 Inventario de Pendientes (TODOs)

Solo **3 marcadores** en todo el código (excl. `venv`), los 3 textos de UI en `streamlit_app.py` (L463, L867, L1200) — no es deuda de código. Codebase limpio de marcadores `TODO` / `FIXME` / `HACK` / `XXX` .

2.7 Features Incompletas

Ítem	Ubicación	Estado
Página Contexto Global	<code>pages/3_🌐_Contexto_Global.py</code>	Stub "en construcción"
Página Energía	<code>pages/4_⚡_Energia.py</code>	Stub "en construcción"
Página Data Hub	<code>pages/5_📊_Data_Hub.py</code>	Stub "en construcción"
Finanzas corporativas	<code>scripts/read_excel_models.py</code> → <code>project_financials</code>	Ingestor construido, DB vacía (0 filas)
Persistencia de proyecciones	<code>projections/models.py</code>	0 filas <code>PROJECTION</code> ; se calculan en vivo
Análisis IA de anomalías	<code>ai_engine/analyzer.py</code>	Implementado, 1 sola ejecución registrada

3. Master Action Plan — Ruta Crítica hacia Producción

- > MoSCoW: **Must** / **Should** / **Could** / **Won't**. Esfuerzo: $S(\leq 2\text{ h}) \cdot M(1/2\text{-}1\text{ día}) \cdot L(2\text{-}3\text{ días})$.
- > Cada tarea está redactada para convertirse en un prompt ONESHOT ejecutable.

Fase 1 — Bloqueantes (correr/desplegar limpio)

ID	Tarea	Archivo/Módulo	MoSCoW	Esf.	Depende
B1	Diagnosticar por qué la ingesta se detuvo el 2026-04-22: revisar runs de GitHub Actions y correr <code>python scripts/backfill.py --days 7</code> para ver si las APIs responden y dónde escribe (sqlite local vs Supabase)	<code>.github/workflows/data_ingestion.yml</code> , <code>scripts/backfill.py</code> , <code>models/db.py</code>	Must	M	—
B2	Reparar el pipeline: asegurar que el cron escribe en la DB de producción (Supabase) con los secrets correctos y vuelve a refrescar a diario	<code>.github/workflows/data_ingestion.yml</code> , <code>data/agent.py</code>	Must	M	B1
B3	Fijar el endpoint canónico (decidir monolito <code>streamlit_app.py</code> vs multipágina <code>🏠_Home.py</code>) y declararlo para Streamlit Cloud	<code>streamlit_app.py</code> , <code>🏠_Home.py</code> , <code>.streamlit/</code>	Must	S	—

Fase 2 — Features Faltantes (mínimo de alcance)

ID	Tarea	Archivo/Módulo	MoSCoW	Esf.	Depende
F1	Resolver las 3 páginas stub: completarlas con la lógica del monolito o eliminarlas si se mantiene el monolito	<code>pages/3_*.py</code> , <code>pages/4_*.py</code> , <code>pages/5_*.py</code>	Must	M	B3
F2	Poblar <code>project_financials</code> ejecutando el ingestor de Excel (si el tab de finanzas corporativas es alcance), o desactivar el tab	<code>scripts/read_excel_models.py</code> , <code>models/schema.py</code>	Should	M	—
F3	Integrar <code>imf_weo.py</code> al patrón de <code>registry.py</code> (adaptar a <code>BaseConnector</code> / <code>fetch_series</code>) o documentar su uso aparte	<code>connectors/imf_weo.py</code> , <code>connectors/registry.py</code>	Could	M	—
F4	Cerrar brechas por proveedor: 5 vars Banxico (token) y 4 vars Ecuador sin datos	<code>connectors/banxico.py</code> , <code>connectors/bce.py</code>	Should	M	B2

Fase 3 — Refactorización y Pruebas

ID	Tarea	Archivo/Módulo	MoSCoW	Esf.	Depende
R1	Eliminar el stack Django legacy (apps <code>core</code> / <code>scraper</code> / <code>ai_engine</code> , <code>config/</code> Django, <code>manage.py</code> , <code>templates/</code> , <code>static/</code> , <code>scheduler/</code>) tras rescatar lo útil	árbol Django	Should	L	B3
R2	Consolidar a un único ORM (SQLAlchemy) e introducir migraciones formales (Alembic)	<code>models/</code> , nueva carpeta <code>migrations/</code>	Should	L	R1
R3	Añadir suite mínima de tests (pytest): parsing de conectores, queries de <code>data/database.py</code> , <code>forecast_ensemble</code>	nuevo <code>tests/</code> , <code>pytest.ini</code>	Must	M	—
R4	Limpiar <code>api_provider = NULL</code> en las 103 variables (etiquetar procedencia)	<code>scripts/seed_variables_v4.py</code> , <code>db.sqlite3</code>	Could	S	—

Fase 4 — Seguridad y Hardening

ID	Tarea	Archivo/Módulo	MoSCoW	Esf.	Depende
H1	Añadir a <code>.gitignore</code> : <code>db.sqlite3</code> , <code>*.sqlite3</code> , <code>memoria_chat.txt</code> ; <code>git rm --cached</code> de los tres	<code>.gitignore</code>	Must	S	—
H2	(Opcional) Purgar <code>db.sqlite3</code> / <code>memoria_chat.txt</code> del historial con <code>git filter-repo</code> /BFG si se considera sensible	repo	Could	M	H1
H3	Parametrizar credenciales de <code>docker-compose.yml</code> con <code>\${VAR}</code> o eliminar el archivo si Docker/Celery queda descartado	<code>docker-compose.yml</code>	Must	S	R1
H4	Rotar la credencial de Supabase (precaución) y confirmar que prod lee de <code>st.secrets</code> /env	Supabase, <code>.streamlit/secrets.toml</code>	Should	S	—
H5	Endurecer <code>.streamlit/config.toml</code> (<code>enableXsrfProtection=true</code>) si el dashboard será público	<code>.streamlit/config.toml</code>	Should	S	B3
H6	Sincronizar <code>.env.example</code> con las vars reales (o eliminar las de Django si R1 se ejecuta)	<code>.env.example</code>	Should	S	R1

Fase 5 — Despliegue / Lanzamiento

ID	Tarea	Archivo/Módulo	MoSCoW	Esf.	Depende
D1	Confirmar Streamlit Cloud como target, configurar secrets en el panel y alinear <code>requirements.txt</code> al stack vivo	<code>requirements.txt</code> , Streamlit Cloud	Must	S	B3, F1
D2	Ampliar CI: <code>ruff</code> + <code>pytest</code> + <code>pip-audit</code> en PR, y notificación si el backfill falla (se cayó silenciosamente en abril)	<code>.github/workflows/</code>	Should	M	R3
D3	Fijar dependencias / lockfile (<code>pip freeze</code> → <code>requirements.lock</code> o pin de versiones)	<code>requirements.txt</code>	Should	S	—
D4	Actualizar documentación obsoleta (<code>docs/audit_report.md</code> , README "Estado del Proyecto") con la cobertura real	<code>docs/</code> , <code>README.md</code>	Could	S	—

4. Definition of Done

- ☐ El cron de ingesta corre a diario y la frescura de `REAL_OFFICIAL` es ≤ 7 días.
- ☐ Un único endpoint Streamlit, desplegado y accesible por URL (Streamlit Cloud) con secrets en el panel.
- ☐ No quedan páginas "en construcción": las 5 funcionan o se consolidan en el monolito.
- ☐ Stack Django legacy eliminado (o explícitamente conservado y documentado); un solo ORM.
- ☐ `db.sqlite3` y `memoria_chat.txt` fuera del repo; sin credenciales hardcodedas en archivos versionados.
- ☐ Suite de tests mínima en verde + `ruff` sin errores, ambos en CI.
- ☐ `requirements.txt` alineado al stack vivo y con versiones reproducibles.
- ☐ Documentación (`README` , `docs/`) refleja la cobertura y arquitectura reales.

5. Siguietes Pasos Inmediatos (propuestos, NO ejecutados)

1. **Diagnosticar el pipeline caído (B1):** `gh run list --workflow=data_ingestion.yml` para ver los últimos runs, y `python scripts/backfill.py --days 7` (con permiso) para comprobar si las APIs responden y por qué se detuvo el 2026-04-22.
2. **Higiene de repo (H1):** en una rama nueva, `git rm --cached db.sqlite3 memoria_chat.txt` y añadir `db.sqlite3`, `*.sqlite3`, `memoria_chat.txt` a `.gitignore`.
3. **Decidir endpoint y plan de limpieza Django (B3 + R1):** confirmar `streamlit_app.py` vs multipágina y planear el `git rm -r` del árbol Django legacy.

Generado por auditoría técnica de solo lectura. La auditoría no modificó ningún archivo del proyecto; esta entrega solo crea archivos nuevos en `docs/`.